



MINISTÉRIO DA EDUCAÇÃO
SECRETARIA DE EDUCAÇÃO PROFISSIONAL E TECNOLÓGICA
INSTITUTO FEDERAL DO NORTE DE MINAS GERAIS – IFNMG
Criado pela Lei no.11.892, de 29/12/2008
Campus Montes Claros



JOÃO KENNEDY SOUZA SOARES

RELATÓRIO : TRABALHO PRÁTICO 1A

Relatório apresentado ao Professor Wagner Ferreira de Barros como parte das exigências de avaliação da disciplina de Processamento Digital de Imagens do Curso de Bacharelado em Ciência da Computação do Instituto Federal do Norte de Minas Gerais, Campus Montes Claros.

MONTES CLAROS - MG

2026

Relatório de Desenvolvimento: Processamento Digital de Imagens com NumPy

O Processamento Digital de Imagens (PDI) é a área da ciência da computação focada na manipulação e análise de imagens através de algoritmos matemáticos. Uma das ferramentas fundamentais para esse tipo de processamento em Python é a biblioteca NumPy, que permite lidar com imagens estruturadas como matrizes multidimensionais.

Diferente de abordagens tradicionais que avaliam pixel por pixel através de laços de repetição, o uso do NumPy permite acessar operações vetorizadas de baixo nível, processando blocos inteiros de matrizes de forma simultânea. Este trabalho foca na aplicação prática desses conceitos para a criação, transformação e mescla de imagens utilizando exclusivamente fatiamento, operações lógicas e estatísticas básicas.

2. Visão Geral e Objetivos do Projeto

O objetivo central deste trabalho foi desenvolver diversas manipulações de imagens utilizando a linguagem Python e a biblioteca NumPy, respeitando a restrição estrita de não utilizar laços de repetição (for ou while).

As funcionalidades principais implementadas incluem:

- Renderização e geração de máscaras e degradês utilizando “np.zeros”, “np.linspace” e fatiamento direto de matrizes.
- Manipulação do espaço de cor RGB, realizando a separação, inversão e anulação de canais de cor específicos.
- Uso de cálculos estatísticos nativos “np.mean” para converter imagens coloridas em tons de cinza e extração de limiares.
- Criação de um efeito "Chroma Key" realista, mesclando diferentes fotografias a partir da geração de matrizes lógicas (booleanas) de cor.

3. Desenvolvimento e Tecnologias

Para a construção do projeto, as seguintes ferramentas e bibliotecas integradas foram utilizadas:

- Linguagem de Programação: Python 3, escolhida por sua sintaxe direta e vasto ecossistema científico.
- Biblioteca Matemática: NumPy foi utilizada para todos os comandos de cálculo estrutural, fatiamento, cópia de arrays e operações condicionais em massa.
- Visualização: Matplotlib foi empregue para a leitura de arquivos de imagem e a renderização dos resultados gráficos na tela.
- Ambiente de Execução: A estrutura do código foi consolidada em um Jupyter Notebook, permitindo a avaliação em blocos lógicos executáveis, facilitando a demonstração dos passos solicitados de forma sequencial.

3. Implementação dos conceitos de Processamento de Imagens

3.1 Fatiamento e Mescla de Matrizes

No projeto, a criação das formas básicas foi realizada utilizando comandos de alocação fixa do NumPy. Para a construção da máscara quadrada na 1.A, uma matriz de base 200x200 foi zerada, e as coordenadas do centro foram reatribuídas diretamente.

Para o degradê horizontal na 1.B, utilizou-se o “np.linspace” para mapear os 255 tons em uma linha, e o comando “np.tile” para replicar essa linha por toda a altura da matriz, eliminando a dependência de loops.

A mescla de ambas na 1.C foi alcançada com êxito através da função “np.where”, atuando como um filtro de mesclagem seletiva.

3.2 Espaço de Cores RGB e Canais

Na manipulação de imagens reais tanto na 2.B e 2.C, mapeamos o sistema de canais tridimensionais do NumPy. A inversão de matrizes entre Azul e Vermelho foi tratada indexando a última dimensão do Tensor.

Para criar o filtro vermelho puro, a técnica aplicada isolou os canais e atribuiu o valor 0 a toda a extensão das matrizes 1 (Verde) e 2 (Azul), o que simplificou drasticamente a colorização seletiva sem alterar a topografia da imagem original.

3.3 Redução Estatística e Limiarização

O cálculo da imagem monocromática na 3 exigiu achatar os canais de cor baseando-se em sua média. A função “`np.mean(img, axis=2)`” cumpriu este papel processando o peso das cores RGB.

Para criar um mapa de alto contraste (Limiarização Automática), utilizou-se o “`np.mean`” para achar o centro de iluminação de toda a cena (Média Geral). Com esse valor numérico base, a operação “`np.where`” avaliou matricialmente se os pixels ficariam totalmente brancos ou pretos, dependendo de sua posição em relação ao limiar extraído.

3.4 Chroma Key e Operadores Booleanos

Para recriar o efeito de substituição de cenário de TVs (Chroma Key), utilizou-se arrays de verdade e falsidade. A cor exata a ser removida (Céu azul) foi separada aplicando o operador lógico unificado & para encontrar a conjunção de ($R==135$, $G==206$ e $B==235$) na 4.A.

Isso gerou uma máscara de recortes. Com base nessa tela de booleanos, a sobreposição final aplicou a técnica de indexação de array transferindo os pixels da imagem de fundo perfeitamente para as lacunas do recorte, resultando em uma colagem vetorizada.

4. Conclusão

O desenvolvimento deste trabalho permitiu a aplicação prática e aprofundada dos conceitos teóricos de matrizes no Processamento Digital de Imagens. O projeto cumpriu com sucesso todos os seus objetivos e restrições, provando a capacidade fenomenal de performance de instruções vetorizadas usando a biblioteca NumPy.

O pipeline estruturado combinando NumPy para o processamento matemático e o Matplotlib para análise gráfica provou ser uma ferramenta didática muito eficaz para solidificar a compreensão das propriedades dos pixels e seus espaços de cores no ambiente computacional.

5. Git Hub

[Git Hub - Trabalho-1A-PDI](#)